

Milestone 3: Component Interfaces & Integration Contract

This document defines the **strict data interfaces** between the four modules of our Airline Graph-RAG pipeline. To ensure we can work in parallel, every module must strictly adhere to these input/output types.

0. Shared Data Models (The “Contract”)

These are the common data structures (represented as Python `dataclasses`) that we pass between layers.

A. Extracted Information (Input -> Retrieval)

This is what the **Input Ingestion** layer produces and hands off to the **Retrieval** layer.

```
from dataclasses import dataclass
from enum import Enum
from typing import List, Dict, Optional

class IntentType(str, Enum):
    FLIGHT_SEARCH = "FLIGHT_SEARCH"           # "Find flights from ORD to LHR"
    DELAY_ANALYSIS = "DELAY_ANALYSIS"         # "Why is flight 123 always late?"
    SATISFACTION_METRICS = "SATISFACTION"     # "How is the food on 737 Max?"
    ROUTE_STATS = "ROUTE_STATS"              # "What are the busiest routes?"
    UNKNOWN = "UNKNOWN"                      # Fallback

@dataclass
class Entity:
    entity_type: str # "AIRPORT", "FLIGHT_NUM", "DATE", "AIRCRAFT", "METRIC"
    value: str       # "ORD", "BA123", "2023-01-01", "737 Max", "Food"

@dataclass
class ProcessedQuery:
    original_text: str
    intent: IntentType
    entities: List[Entity]
```

B. Graph Context (Retrieval -> LLM)

This is the standard output format for **BOTH** the Graph Retrieval (Cypher) and Embeddings (Vector) layers. The LLM layer will receive a list of these items.

```
class RetrievalSource(str, Enum):
    CYPHER = "CYPHER"
    VECTOR = "VECTOR"
```

```

@dataclass
class ContextChunk:
    id: str          # Unique ID of the node/record (e.g., "Journey_12345")
    text: str        # Human-readable info (e.g., "Journey 123 had 45min delay, Food")
    score: float     # Relevance score (1.0 for Cypher exact match, 0.0-1.0 for Vector)
    source: RetrievalSource # Where did this come from?
    metadata: dict   # Raw props for UI visualization (e.g., {"lat": 50, "lon": -0.1})

```

1. Input Ingestion Module

Owner: [Name] **Goal:** Clean raw text and extract structured intent/entities.

Interface

```

def process_user_query(raw_query: str) -> ProcessedQuery:
    """
    1. Clean text.
    2. Classify Intent (Rule-based or LLM).
    3. Extract Entities (NER).
    """
    pass

```

Example Output

```

ProcessedQuery(
    original_text="Show me bad food ratings on flights from ORD",
    intent=IntentType.SATISFACTION,
    entities=[
        Entity(entity_type="METRIC", value="Food"),
        Entity(entity_type="AIRPORT", value="ORD")
    ]
)

```

2. Graph Retrieval Module (Baseline/Cypher)

Owner: [Name] **Goal:** Execute deterministic Cypher queries based on Intent.

Interface

```

def query_graph_cypher(processed_input: ProcessedQuery) -> List[ContextChunk]:
    """
    1. Map processed_input.intent to a Cypher Template.
    2. Inject processed_input.entities into the template.
    """

```

```

3. Run against Neo4j.
4. Format results as ContextChunks.
"""
pass

```

Internal Logic

- Must maintain a `TEMPLATE_MAP` dictionary (Intent -> Cypher String).
 - **Credentials:** Read from `config.txt` (do not hardcode).
-

3. Embeddings Module (Vector Search)

Owner: [Name] **Goal:** Find semantically similar records using Vector Search.

Interface

```

def query_graph_vector(raw_query: str, k: int = 5) -> List[ContextChunk]:
    """
    1. Generate embedding for raw_query.
    2. Run KNN search against Neo4j Vector Index.
    3. Format results as ContextChunks.
    """
    pass

```

Airline Specifics

- Since our data is numerical, this module assumes we have pre-calculated text descriptions for nodes (e.g., a property `text_repr`: "Flight 101, Delay: High, Food: Low").
 - *Note:* This module is also responsible for the setup script that creates these embeddings if they don't exist.
-

4. LLM & UI Module

Owner: [Name] **Goal:** Synthesize answer and display interface.

Interface

```

def generate_response(user_query: str, context: List[ContextChunk]) -> str:
    """
    1. Deduplicate context chunks (remove same IDs).
    2. Construct Prompt (Persona: Airline Analyst).
    3. Call LLM (GPT/Gemini/Llama).
    4. Return text answer.
    """

```

```

    """
    pass

def render_ui():
    """
    Streamlit App Loop:
    1. Get Input -> call process_user_query().
    2. Parallel Call -> query_graph_cypher() AND query_graph_vector().
    3. Aggregate Results -> context list.
    4. Call generate_response(context).
    5. Display Answer + ContextChunks.
    """
    pass

```

Integration Guide (How to Stub)

To work independently, create a `stubs.py` file in your module.

If you are the LLM person: You don't need the real Graph module yet. Just create a fake function:

```

# stubs.py
def mock_retrieve(query):
    return [
        ContextChunk(
            id="1",
            text="Flight 123 was delayed by 30 mins",
            score=1.0,
            source=RetrievalSource.CYPHER,
            metadata={}
        )
    ]

```

Build your UI using this mock. When the Graph team is ready, swap the import.

If you are the Graph person: You don't need the real Input module. Just create a dummy object:

```

# test_graph.py
dummy_input = ProcessedQuery(
    original_text="...",
    intent=IntentType.FLIGHT_SEARCH,
    entities=[Entity(entity_type="AIRPORT", value="LHR")]
)
print(query_graph_cypher(dummy_input))

```